# Ejercicios Laboratorio 4

Lenguajes de programación y procesadores de lenguaje

20 de Noviembre de 2025

En el Campus Virtual de la asignatura en el apartado de "Laboratorio 4: Análisis de identificadores y tipos" encontraréis un enlace para obtener CUP como archivo `cup.jar` y otro para obtener el código Java para generar el analizador sintáctico en los que vamos a trabajar en la sesión de hoy `constructorAST.zip`. Descargad todos estos archivos en una carpeta y descomprimid los ficheros `.zip`.

Para más información o versiones más recientes de CUP visitad `http://www2.cs.tum.edu/projects/cup/`.

Podéis instalar CUP en vuestra cuenta usando el archivo `.jar` o lo podéis usar directamente en la línea de comandos. También podéis utilizar el entorno para desarrollo de código Java que prefiráis. Deberéis definir los caminos necesarios en el `CLASSPATH` o usar `-cp` al llamar a Java.

## Ejercicio: Generador AST

En la sesión de hoy vamos a introducir dos análisis semánticos típicos al compilador en el que veníamos trabajando en las sesiones anteriores: el análisis de identificadores y el análisis de tipos. Podéis encontrar la implementación en los distintos nodos de nuestro ast:

- La función `bind` realiza el análisis de identificadores. Su función es recorrer todas las instrucciones y expresiones de nuestro programa y asegurarse de que los usos de identificadores se encuentran correctamente definidos. Para cada uso de variable guarda una referencia a la definición de la misma (binding). La función `bind` utiliza una pila de tablas de identificadores (TableStack) para su análisis.
- La función `checkTypes` realiza el análisis de tipos. Su función es recorrer todas las instrucciones y expresiones de nuestro programa, asegurarse de que se encuentran bien tipadas y almacenar su tipo. Para

ello actualiza el atributo `type` de las expresiones, al que posteriormente puede acceder a través del método `getType`.

Los comandos a utilizar para generar y compilar el analizador son similares a los de las prácticas anteriores:

Después para compilar el proyecto hacemos desde el directorio constructor, llamamos a:

```
javac -cp ../cup.jar */*.java
```

Al ejecutar el programa lo que hacemos es realizar ambos análisis. En caso de que el programa sea correcto no devolvemos nada. Si detectamos errores los mostramos por pantalla. En este caso, estamos analizando el AST representando el programa almacenado en input.txt:

```
java -cp ../cup.jar: constructorast.Main  ../input.txt
```

# Modificaciones a realizar

Para practicar, podemos añadir los cambios que hicimos para extender el análisis léxico y sintáctico a nuestros análisis. De cara a añadir los cambios debemos:

1. Actualizar la función bind() de los nodos del ast que hemos generado.
2. Actualizar la función checkTypes() de los nodos del ast que hemos generado.

Algunos cambios que podéis realizar son los siguientes (ordenados de más sencillos a más complejos) son los siguientes (son los mismos en los que trabajamos en la práctica anterior):

- Añadir el operador para la multiplicación
- Añadir otros operadores relacionales y las operaciones entre booleanos
- Añadir instrucciones para permitir bucles
- Añadir números reales y el tipo de número real
- ...

Recuerda que de cara al proyecto final también debéis implementar los análisis extra que habéis obtenido vía carta.

Otro posible cambio a realizar es mejorar los mensajes de error generados añadiendo información acerca de la línea en que se detecta el error. Para ello debemos de añadir la información `linea` como un atributo de nuestros nodos del ast y pasarlo como argumento desde el fichero .cup al crear los nodos (podéis acceder a ella desde el .cup llamando a variable.fila()).